DSCI 551 COURSE PROJECT, SPRING'26 Phase 2:

Midterm Report & Implementation Checkpoint

Group Members

Chanyoung Kim
Yogita Mutyala

Database system overview and architecture

Database System: PostgreSQL 18.1 with Apache AGE 1.7.0.

In the domain of financial fraud detection, transactional data benefits from the ACID guarantees of a relational DBMS, while suspicious money flows often require relationship-oriented analysis. The motivation of this project remains the same as in the proposal: to evaluate whether a graph-on-relational approach can support AML analysis without maintaining separate relational and graph database systems.

At the midterm checkpoint, the implementation runs inside a Docker container named **apache-age**. The system includes relational tables for customers, accounts, and transactions; a synthetic workload generator; a SQL-based smurfing detector; and an AGE graph named **aml_graph** that models account-transfer relationships for subsequent Cypher-based traversal experiments.

This differs from a native graph database because the authoritative transaction data still resides in PostgreSQL heap tables, and it differs from plain PostgreSQL because AGE adds graph labels, Cypher queries, and edge-based traversal to the same workload. At the physical level, PostgreSQL stores relational data in heap-organized tables divided into fixed-size pages, with each transaction record stored as a tuple inside a page. This physical organization influences whether the planner prefers sequential scans

or index-based access paths during fraud detection queries.

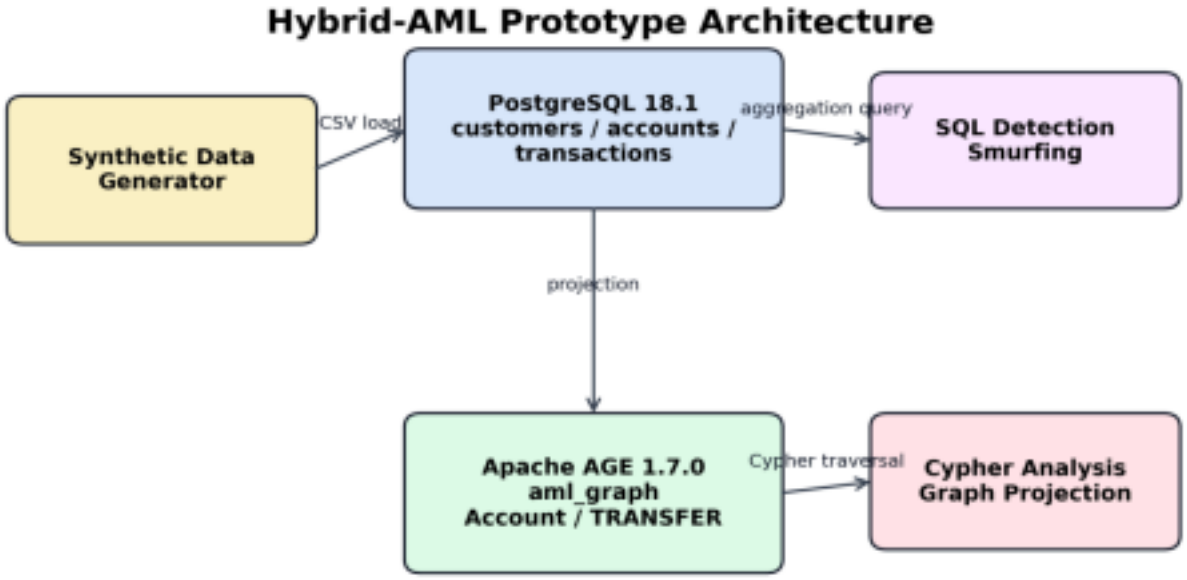

Figure 1: Midterm prototype architecture connecting the synthetic workload, relational tables, and AGE graph projection

Planned Internal Focus Areas and Motivation

Focus Area 1 is storage architecture. At this stage, the relational schema has been implemented, and the same data has been projected into AGE as Account vertices and TRANSFER edges. Apache AGE internally represents graph data using relational tables within the ag_catalog schema. Each vertex and edge label corresponds to a table, and properties are stored using the agtype format. This design is important for our project because it means graph workloads are ultimately executed on top of relational storage rather than through a separate native graph engine. Focus Area 2 is query execution. The current checkpoint demonstrates a working SQL aggregation query for smurfing detection and the successful graph materialization needed for later Cypher-based traversal analysis.

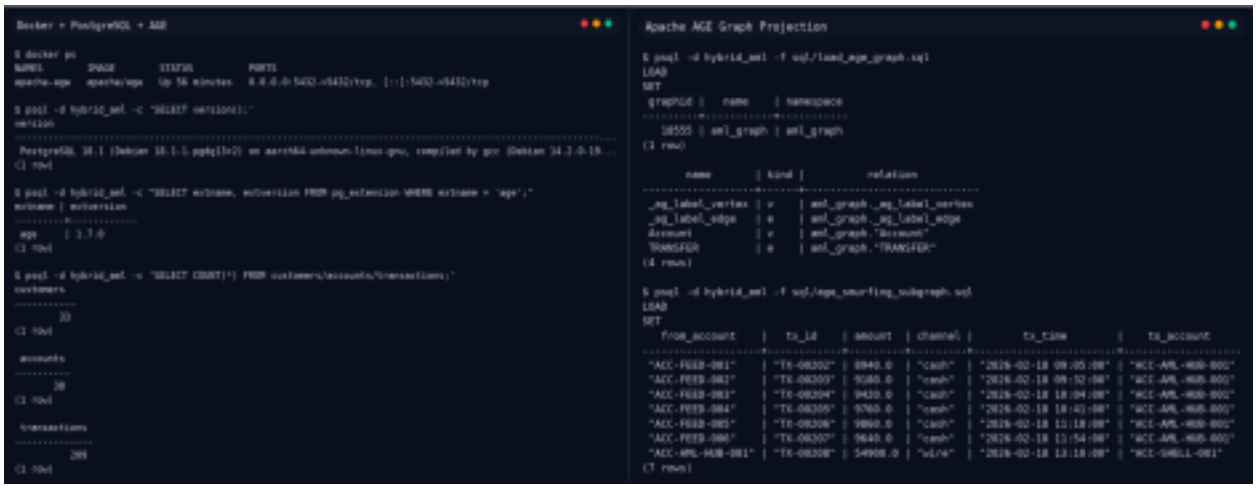

Figure 2: Left: Docker and extension status. Right: AGE graph projection and suspicious-edge query result

Evaluation Metrics

The evaluation metrics remain query latency, planner cost, storage overhead, throughput under a synthetic workload, and index effectiveness. The current checkpoint provides the first latency evidence through SQL execution planning and runtime. The measured planning time for the smurfing query is 0.673 ms, and the measured execution time is 0.139 ms.

Preliminary Application Idea

Application Name: Hybrid-AML

The application remains a fraud-monitoring prototype with two core features from the proposal: smurfing detection and circular trading detection. At the current checkpoint, the implemented feature is smurfing detection. The dataset contains 33 customers, 38 accounts, and 209 transactions, mixing regular payroll, rent, utilities, consumer spending, peer-to-peer transfers, and one explicit smurfing funnel. The suspicious hub account receives six cash deposits totaling less than $10,000 on the same day, then wires most of the total to a shell account. The application pipeline consists of synthetic data generation, relational ingestion, graph projection, and query-based fraud detection.

Figure 3: Flow of Data

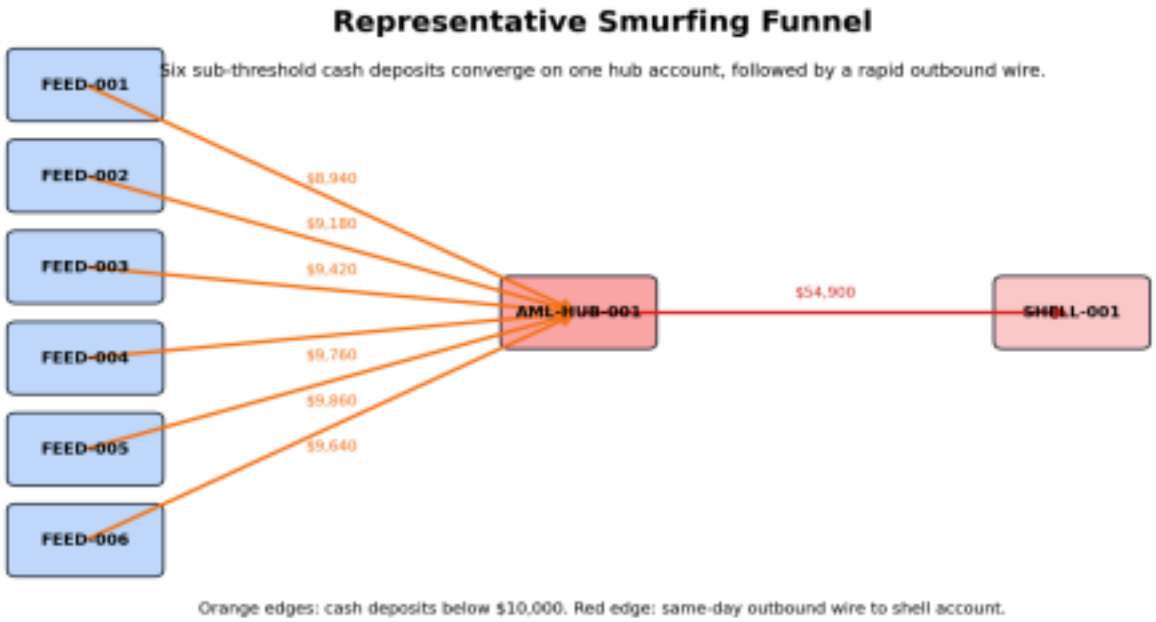

Figure 4: The synthetic smurfing scenario used for the implementation checkpoint.

Implementation Checkpoint

The checkpoint requirements are satisfied. PostgreSQL and AGE are installed and configured in Docker. The initial relational schema is implemented. The synthetic dataset loads successfully. The SQL smurfing detector returns one suspicious case for account **ACC-AML-HUB-001**. The AGE graph projection also loads successfully with 38 account vertices and 209 transfer edges, providing a starting point for the graph-focused second feature.

Checkpoint Item	**Current Status**
Database installed and configured	Completed in a Docker container apache-age (See Figure 2)
Initial schema implemented	customers, accounts, transactions (See Figure 5)
Working feature or query	SQL smurfing detector returns one suspicious funnel (See Figure 6)
Graph setup progress	aml_graph contains projected Account vertices and TRANSFER edges (See Figure 2)

Table 1: Progress on project implementation checkpoint items

Figure 5: Relational Schema: Customers, Accounts, and Transactions

Figure 6: Left: the SQL detector identifies the intended suspicious beneficiary. Right: the execution plan shows a small-table sequential scan followed by an indexed outbound lookup.

Preliminary Mapping Between DB Internals and Application Behaviors

The smurfing detector illustrates how relational internals drive application behavior. The query first scans the transactions table to isolate cash deposits and groups them by beneficiary account and day. Because the current dataset is small, the planner chooses a sequential scan on the deposit-filter stage. Once a suspicious deposit cluster is found, PostgreSQL uses the composite index on **transactions(from_account_id, tx_time)** to find rapid outbound transfers from the same account. The measured planning time is 0.673 ms, and the measured execution time is 0.139 ms. This is a useful checkpoint result because it shows the exact place where relational indexing helps the funnel-detection workflow. As the dataset grows, sequential scans will become inefficient, and index usage will dominate performance. This highlights how database internals directly impact the scalability of fraud detection workflows.

The AGE graph projection provides the second mapping. Each account becomes an Account vertex, and each transaction becomes a TRANSFER edge with properties such as amount, channel, transaction time,

and fraud label. This projection does not replace the relational schema; instead, it allows the same transaction stream to be analyzed later through Cypher traversals. That design directly matches the proposal's graph-on-relational theme.

At the current stage, these results do not show that the graph layer is better or faster than the relational approach. The midterm evidence only establishes that the relational feature works and that the graph projection has been loaded successfully. A fair comparison will require circular-trading workloads and side-by-side Cypher-versus-SQL measurements in the next phase.

Progress Update and Challenges

Progress since the proposal is concrete but intentionally scoped. The Docker environment is stable, the relational schema is implemented, the synthetic dataset is reproducible, the smurfing detection feature works, and the initial AGE graph is loaded. The main unfinished item is circular trading detection, which requires generating graph patterns that contain cycles and then comparing Cypher traversal behavior against equivalent SQL logic.

The main challenges are Apache AGE query tuning, the small size of the current synthetic dataset, and the need to keep the report focused on results that are already reproducible. In addition, the next phase must account for Cypher-to-SQL translation overhead, possible join expansion in traversal-style workloads, and the difficulty of evaluating graph-query performance fairly when the workload size is still limited. The next phase will add circular-trading scenarios, increase the workload size, collect more EXPLAIN ANALYZE evidence, and compare graph-oriented traversal costs against native relational formulations.